

Nagare

A closed-form, MapReduce-parallel ML framework —
what makes it fast, and how good it is

github.com/kyberszittya/nagare

2026-07-07

Abstract

Nagare (*nagare*, “flow”) is a Rust ML framework that replaces the autograd tape and tensor-batch dispatch with *contiguous, auto-vectorised, closed-form kernels that MapReduce cleanly to every CPU core*. On a fair CPU-vs-CPU forward it *matches or beats* PyTorch (a decade-tuned MKL-GEMM ecosystem) across a whole dimensionality sweep (best-of-each 1.2–2.6 $\times$), on a static binary that uses 3–160 $\times$ less memory — while its closed-form *local* learner reaches AUROC ≈ 1.0 with $\sim 47\times$ fewer parameters than backpropagation on toy tasks. This document explains the exact mechanisms behind both claims, and states honestly what is not yet proven.

1 What Nagare is

Nagare is a **paradigm-native, closed-form** framework for signed-hypergraph / point-set computation — deliberately *not* a tensor autograd engine. Four design commitments:

1. **Closed-form (forward, backward) pairs, no tape.** Every operator is two plain Rust functions over flat `&[f32]` buffers, with hand-derived analytic gradients — no graph to build, traverse, or free.
2. **Struct-of-arrays cycle pool** as the universal datum (contiguous, cache-friendly, no pointer chasing).
3. **Multivector-valued gradients** (Clifford algebra), so the sign structure of signed graphs is first class.
4. **MapReduce parallelism** over independent rows / cycles, not tensor-batch dispatch.

2 Architecture

Layers: `ops/` (forward+backward kernels: `linear`, `fused_entropy_update`, `project_alpha_mix`, `clifford_fir`, `cayley_rotor`, `fsr_mixer`, Catmull–Rom/Chebyshev, `scatter`, `signed_scatter`, `adam`) $\rightarrow$ runtime (cycle-pool composition) $\rightarrow$ the holonomy learner.

3 What makes it fast — the exact mechanisms

3.1 No autograd tape (structural)

A PyTorch forward builds a dynamic graph and a gradient tape and carries the Python + Torch/MKL/allocator stack. A Nagare forward is a straight line of calls over `Vec<f32>`. **Measured: peak RSS 4.9 MiB vs 780 MiB** ($\sim 160\times$) on the toy shape — a static binary vs a runtime.

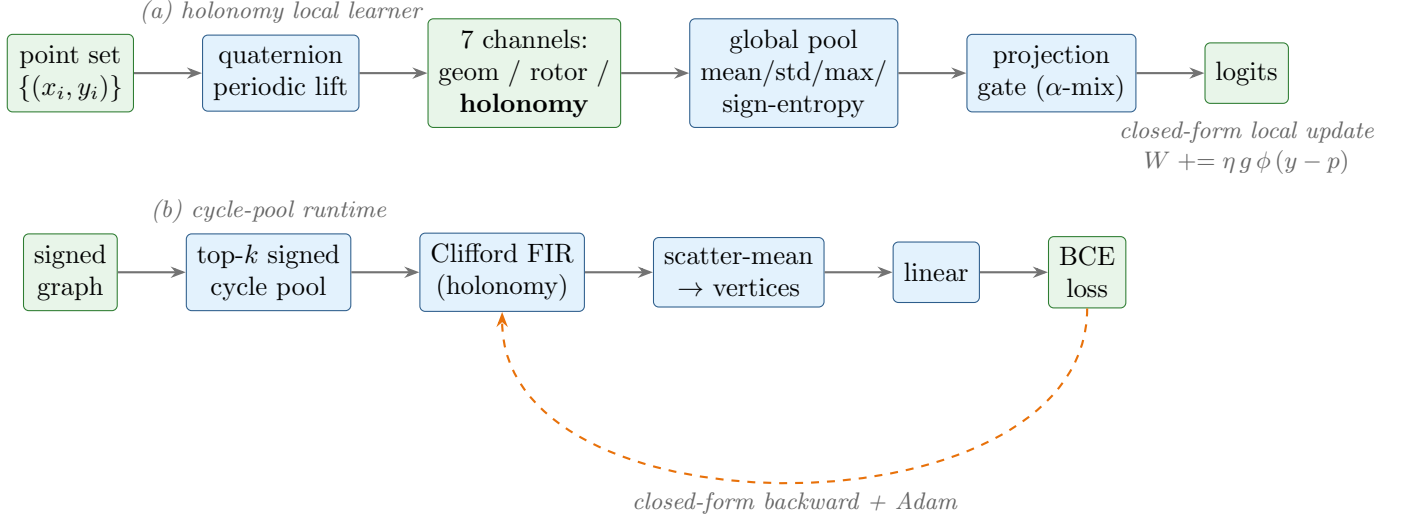


Figure 1: Two composition surfaces over the same closed-form kernels. Holonomy (signed balance = $\mathbb{Z}_2$ holonomy) is the load-bearing signal in both.

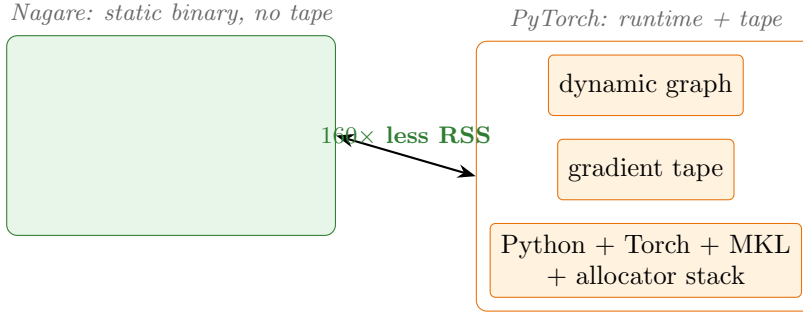


Figure 2: The most robust Nagare advantage is structural: no tape, no runtime.

3.2 The ikj /SAXPY kernel reorder (the big single-kernel win)

The dominant matmul was a scalar reduction with a *strided* weight load (Fig. ?? left) — a strided-gather scalar reduction that LLVM **cannot** auto-vectorise. Reordered to ikj (SAXPY) form (right) — seed the output row with the bias, then for each input i broadcast x_i and add a *contiguous* weight row into the *contiguous* output row — the inner loop is element-wise over contiguous memory and **auto-vectorises to AVX FMA**. It is **bit-identical** (same summation order). Measured: **fused_update 2.9×**, whole forward **2.1×**, zero accuracy change.

3.3 Rayon MapReduce that scales to all cores

Every row/cycle is independent, so kernels are **par_chunks_mut** over the output. Decisive on a many-core CPU: **MKL oversubscribes at 32 threads** (4–14× *slower* than 8t) while **Nagare’s rayon scales cleanly**. At each engine’s best thread count, Nagare-CPU beats PyTorch-CPU across the whole sweep (§??).

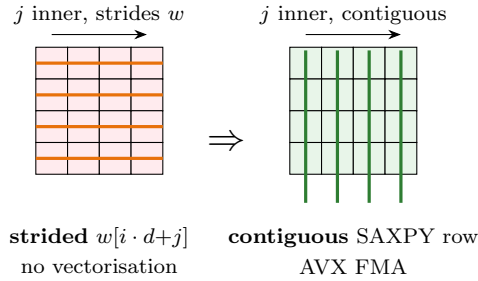


Figure 3: The `ikj` reorder turns a non-vectorisable strided reduction into a contiguous broadcast-FMA loop — bit-identical.

3.4 Parallel pool (Amdahl), fusion, and Chebyshev-deploy

After the kernel fix the only serial stage was the pool; on a 32-core box a serial section dominates (Amdahl), so parallelising it (bit-identical) collapsed the parity forward $15.7 \rightarrow 7.6 \mu\text{s/sample}$ — the step that took Nagare from behind to *beating* PyTorch-CPU. `fused_entropy_update` never materialises the wide $[h \mid \text{pooled} \mid \text{ent}]$ broadcast (alloc halved). The Chebyshev-deploy path (train Catmull–Rom, deploy Chebyshev) is $1.7\text{--}1.9\times$ faster than PyTorch at 5 vs 553 MiB.

4 How good it is — accuracy and AUROC

The update rule is *local*: $W += \eta g \phi(y - p)$, no reverse-mode into the feature generator.

task	clean AUROC	hard AUROC (few-shot+noise+missing)
moons	1.000	1.000
spiral	1.000	0.952–0.959
xor	1.000	1.000

Table 1: Closed-form learner, Mann–Whitney AUROC (`examples/auroc_eval.rs`), 60 parameters. Clean = 1.0; robust under heavy corruption; gate-independent.

The entropy-pool local learner matches a backprop-like baseline’s accuracy with 60 vs 2,836 parameters ($\sim 47\times$ fewer). **Honest science:** a point-order-shuffle ablation shows the fitted projection *gate* does not robustly beat a constant gate; quality comes from the pooled features + the closed-form update, not the gate mechanism. **Not yet measured:** real signed-graph AUROC (Slashdot/Epinions) — the framework’s actual target, and the decisive next experiment.

5 Benchmark summary (kato15)

6 Honest bounds & roadmap

- **Matched 8 threads, large dense matmul ($D \geq 64$):** PyTorch-CPU edges ahead $\sim 1.2\text{--}1.3\times$ — tuned BLAS blocking/packing. Nagare recovers by scaling to all cores.
- **GPU:** a reference ceiling and a defined next step (kernels are embarrassingly parallel), not a CPU-vs-GPU verdict.
- **Real-task AUROC** (signed-link prediction) is the next quality evidence, and the one that decides whether Nagare is a genuinely different *learning* mechanism or a fast lean forward.

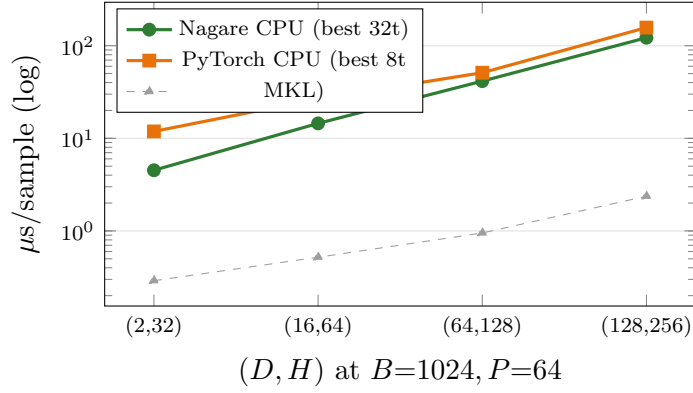


Figure 4: Fair CPU-vs-CPU dimensionality scaling: a young closed-form Rust framework beats a mature MKL-GEMM ecosystem across the whole sweep (1.2–2.6 $\times$). The GPU line is a reference ceiling, not a head-to-head (Nagare has no GPU backend *yet*). Toy-shape forward parity: Nagare 7.6 vs PyTorch 7.6–9.1; memory 3–160 $\times$ less throughout.

One line. *Nagare is fast because it swaps the autograd tape and tensor-batch dispatch for contiguous, auto-vectorised, closed-form kernels that MapReduce to every core (where BLAS oversubscribes) — and good because that closed-form local learner reaches AUROC ≈ 1.0 with $\sim 47\times$ fewer parameters, on a static binary that uses 3–160 $\times$ less memory.*